Infrastructure Orchestration & Automation — Bloc 3

Jour 3 — Infrastructure as Code

Terraform et l'automatisation par pipeline CI/CD.

Enseignant
Dr. Abdelhak TOUITI

Matin
Cours — 9h00 à 12h30

Après-midi
TP guidé — 13h30 à 17h00

Outils
Terraform GitLab CI

Compétences
C8.2 · C7.1

Programme

Objectifs de la journée

- Comprendre les principes de l'Infrastructure as Code : déclaratif, versionné, reproductible.
- Provisionner une infrastructure complète (plusieurs ressources liées) avec **Terraform**.
- Comprendre le graphe de dépendances que Terraform construit automatiquement.
- Automatiser la validation du code d'infrastructure avec un pipeline **GitLab CI/CD** à plusieurs étapes.

Programme

Déroulé de la journée

Matin — Cours

- Principes de l'Infrastructure as Code
- Terraform : HCL, state, providers, graphe de dépendances
- Le workflow init / plan / apply
- Automatiser avec un pipeline GitLab CI/CD à plusieurs étapes

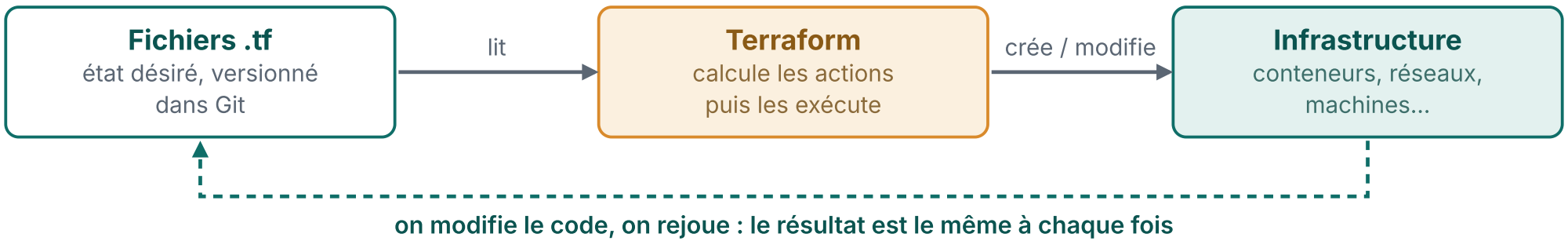
Après-midi — TP guidé

- Provisionner plusieurs ressources liées avec Terraform
- Explorer le graphe de dépendances généré
- Mettre en place un pipeline GitLab CI à plusieurs stages, avec variables
- Versionner l'ensemble dans Git

Définitions

Qu'est-ce que l'Infrastructure as Code ?

Décrire l'infrastructure sous forme de fichiers texte versionnés, plutôt que de la configurer à la main.



- Un fichier décrit l'**état désiré** ; le même fichier recrée un environnement identique.
- Les changements passent par Git : historique, revue, retour en arrière possible.

Définitions

Déclaratif vs impératif

Impératif : les étapes

À vous de gérer chaque cas (déjà créé ? en échec ?).

```
# à exécuter dans l'ordre
docker network create tp3
docker pull nginx
docker run -d --name web nginx
# relancé : « name already in use »
```

vs

Déclaratif : le résultat

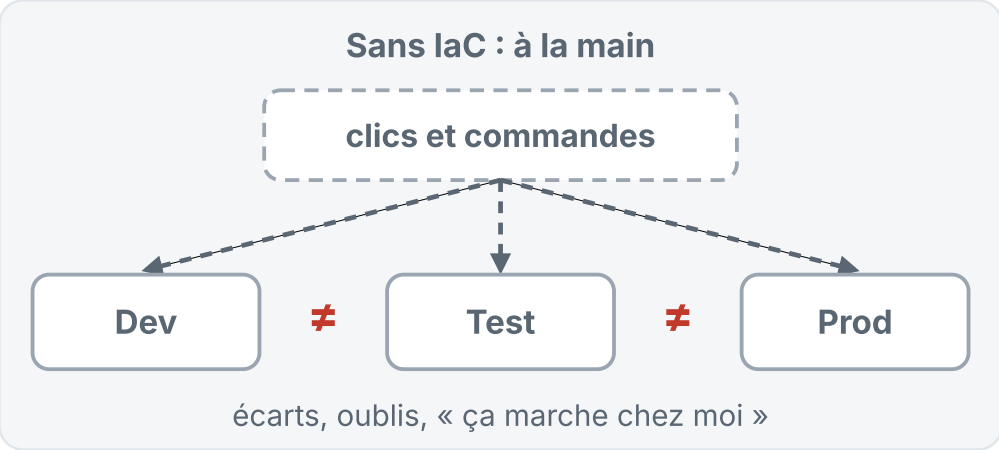
L'outil déduit les actions à mener.

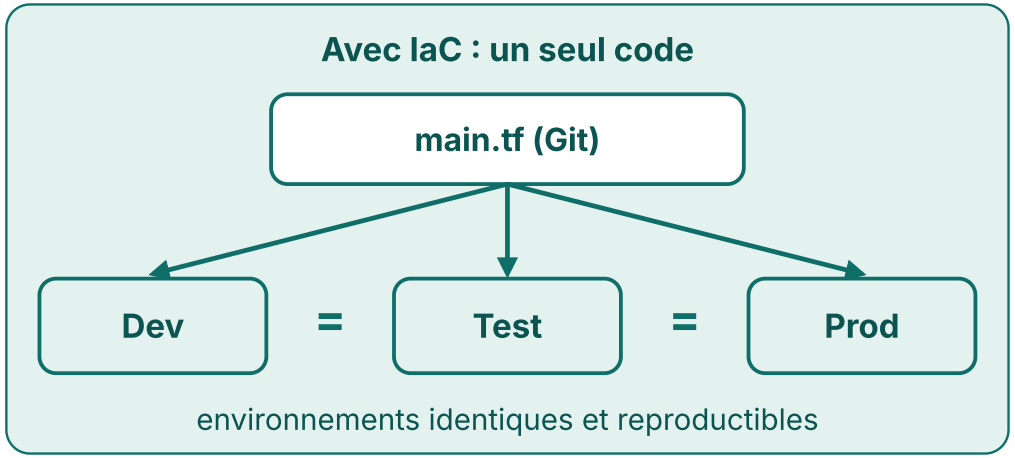
```
# l'état voulu, rien d'autre
resource "docker_container" "web" {
  name  = "tp3-web"
  image = "nginx"
}
# relancé : « No changes »
```

Terraform est déclaratif : on peut rejouer le code sans danger, quel que soit l'état de départ.

Enjeux

Pourquoi l'IaC ?

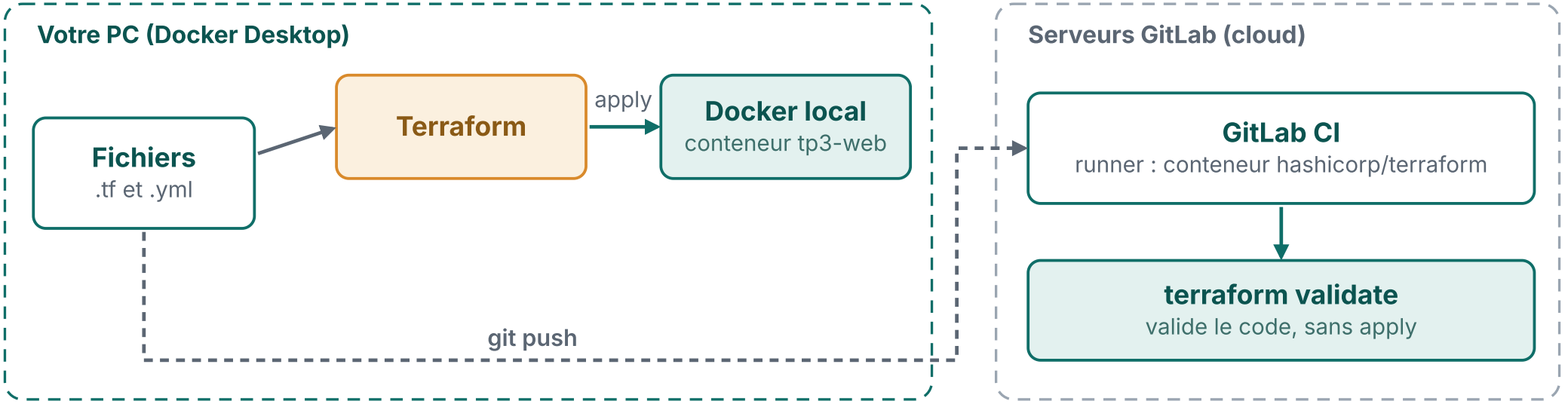




- **Reproductibilité** : même code, même résultat.
- **Traçabilité** : chaque changement est un commit Git.
- **Revue** : l'infra se relit comme du code.
- **Rapidité** : un environnement en quelques minutes.

Cadrage

Les deux rôles de la journée



Terraform agit sur votre machine ; GitLab CI tourne dans le cloud et ne fait que valider le code poussé.

Module



Provisionner avec Terraform

Décrire l'infrastructure voulue, et laisser Terraform calculer comment l'obtenir.

Terraform

Qu'est-ce que Terraform ?

Un outil d'Infrastructure as Code développé par HashiCorp, qui provisionne des ressources chez de nombreux fournisseurs (cloud, Docker, DNS…) à partir d'un code déclaratif.

- Écrit en **HCL** (HashiCorp Configuration Language), un langage dédié, lisible.
- **Multi-provider** : AWS, Azure, GCP, mais aussi Docker, Kubernetes, GitHub…
- Licence **BSL** depuis 2023 — gratuit à l'usage, mais plus formellement open source.

Terraform

La structure d'un bloc Terraform

```
resource "docker_container" "web" {
  name  = "tp3-web"
  image = docker_image.nginx.image_id
}
```

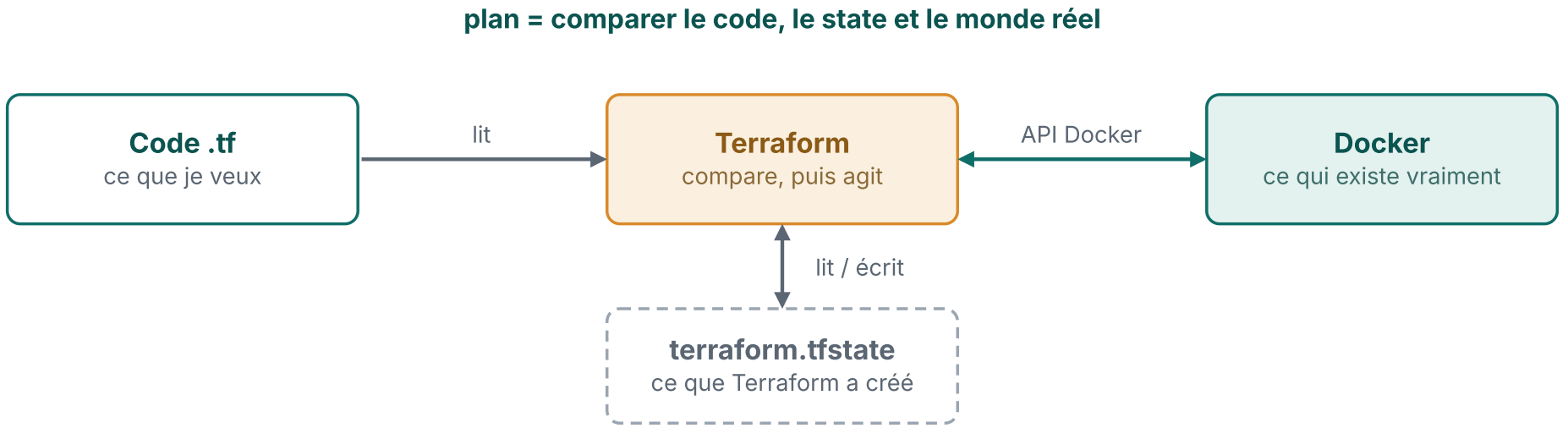
- **resource** : mot-clé, « je déclare une ressource ».
- **type** fourni par le provider : il fixe les arguments possibles.
- **nom local** : sert à référencer la ressource ailleurs.
- **référence** à une autre ressource : Terraform en déduit l'ordre de création.

Anatomie d'une référence

docker_image	.	nginx	.	image_id
type de la ressource référencée		son nom local		attribut qu'elle expose une fois créée

Le state

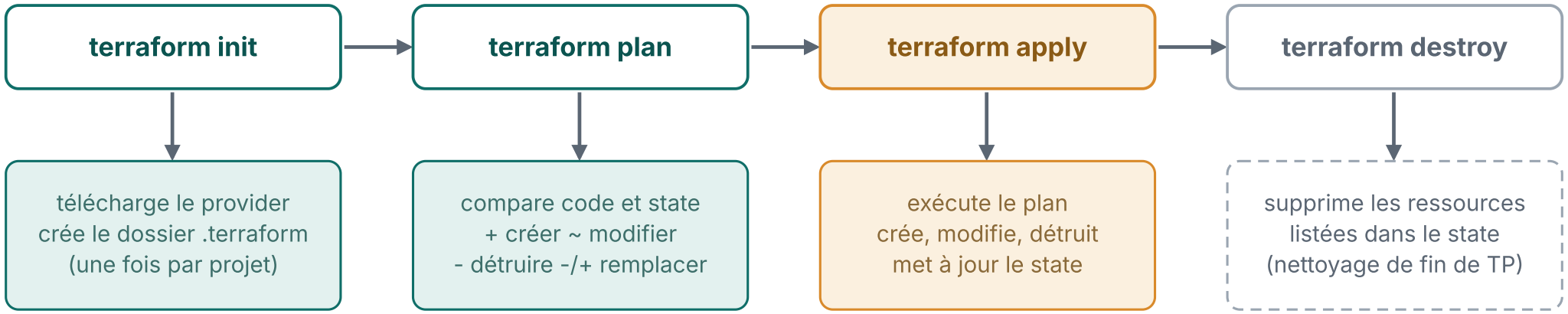
Terraform garde dans `terraform.tfstate` la correspondance entre votre code et les ressources créées.



- Il ne se modifie **jamais** à la main, et ne se committe pas en clair dans Git (données sensibles possibles).

Terraform

Le workflow : init, plan, apply

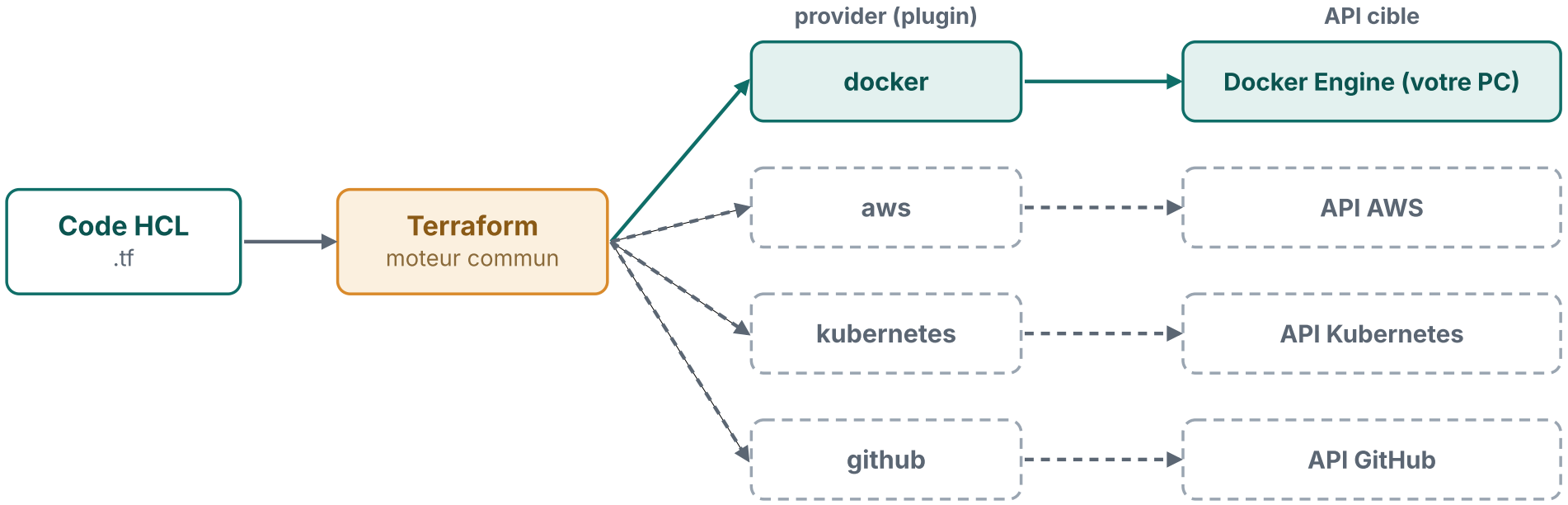


`plan` ne modifie rien : c'est votre filet de sécurité avant `apply`.

Terraform

Les providers

Un **provider** traduit le code HCL en appels à l'API d'un fournisseur : c'est ce qui rend Terraform multi-cible.

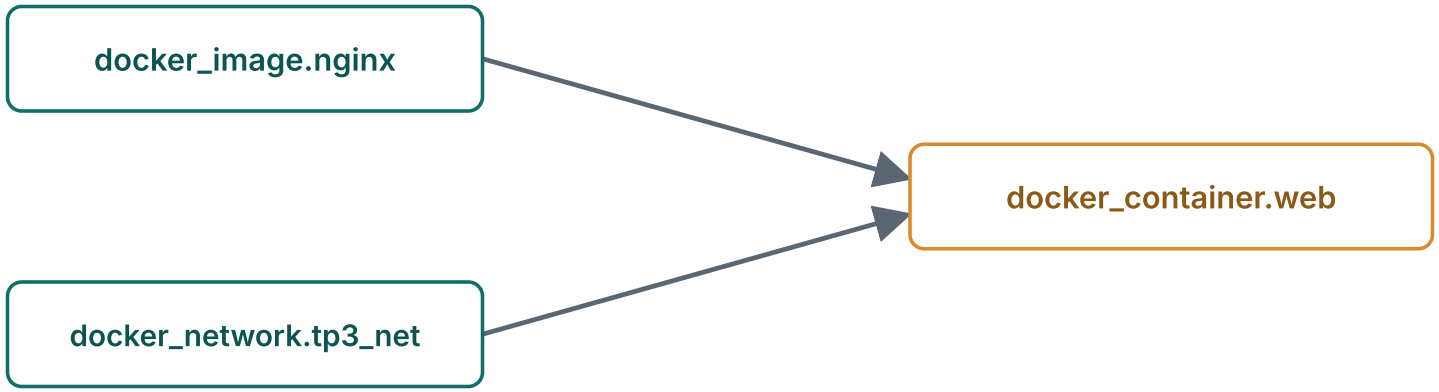


Même langage, même workflow : seul le provider change. Cet après-midi : Docker, sans compte.

Terraform

Le graphe de dépendances

Quand une ressource référence une autre, Terraform construit automatiquement un graphe — exactement le même principe que le DAG Airflow du Jour 1.



- Vous ne déclarez jamais l'ordre vous-même : Terraform le déduit des références entre blocs.
- `terraform graph` affiche ce graphe en ligne de commande.

Module



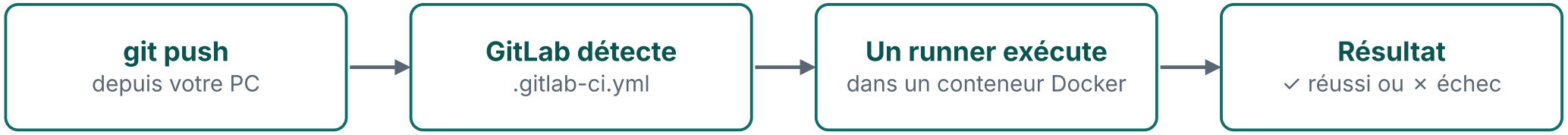
Automatiser avec GitLab CI/CD

Déclencher une validation automatique à chaque modification du code d'infrastructure.

GitLab CI/CD

Un pipeline pour l'infrastructure

Le fichier `.gitlab-ci.yml`, versionné avec le code, déclenche des jobs à chaque `git push`.

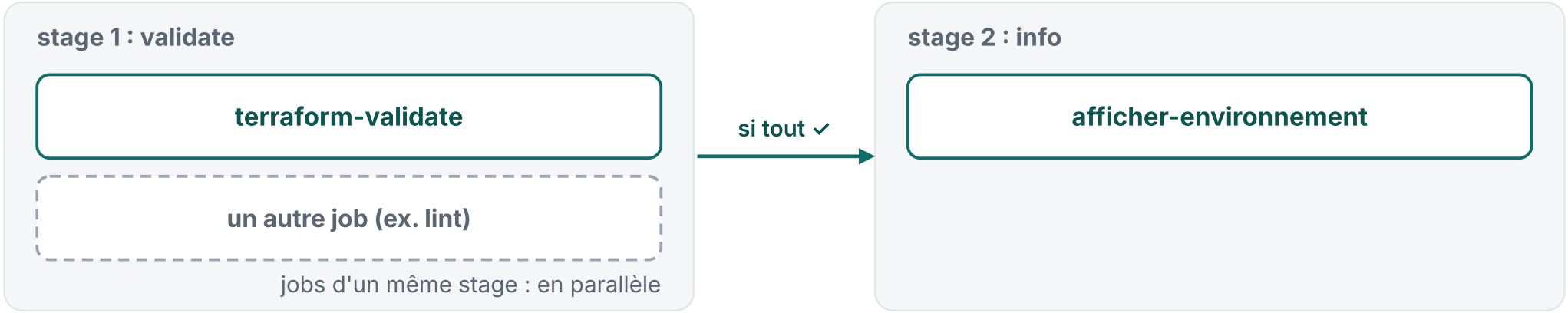



```
stages:
  - validate
terraform-validate:
  stage: validate
  image: hashicorp/terraform:latest
  script:
    - terraform init -backend=false
    - terraform validate
```

GitLab CI/CD

Stages et jobs

Un pipeline est une suite de **stages** ; chaque stage contient un ou plusieurs **jobs**.



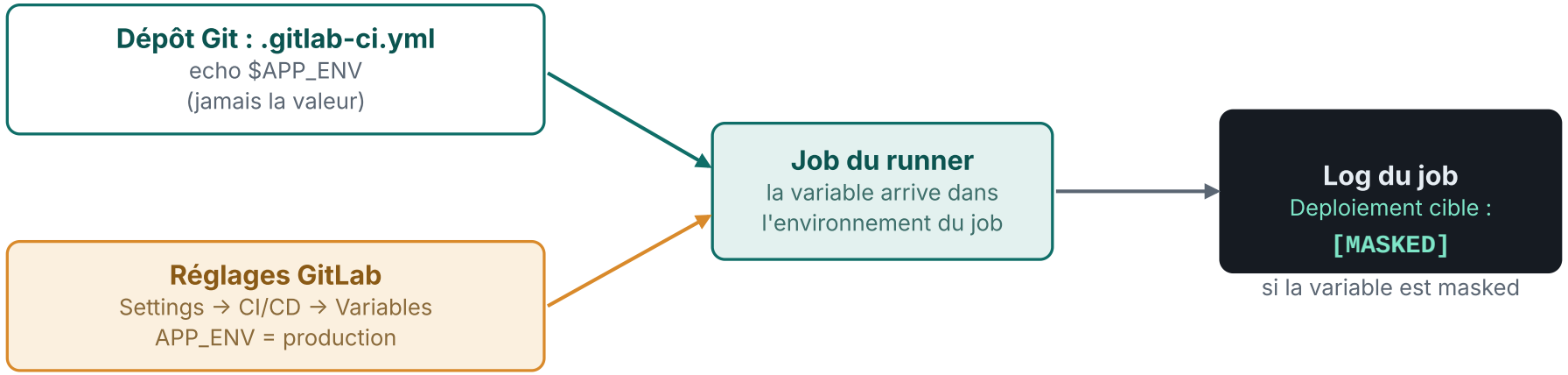
× dans un job : les stages suivants ne sont pas lancés

- Chaque job tourne dans son propre conteneur Docker (`image:`), isolé des autres.

GitLab CI/CD

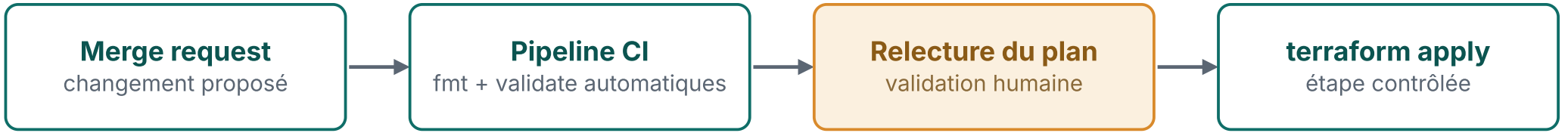
Variables et secrets CI/CD

Les valeurs sensibles ne figurent jamais dans `.gitlab-ci.yml` : on les déclare dans l'interface GitLab.



- Une variable **masked** n'apparaît jamais en clair dans les logs (valeur d'au moins 8 caractères).
- Une variable **protected** n'est disponible que sur certaines branches (ex. `main`).

Bonnes pratiques & risques



- Ne **jamais** committer un secret (mot de passe, clé API) dans le code ou le state.
- Relire un `terraform plan` avant d'appliquer, surtout en production.
- La CI valide automatiquement ; l'`apply` reste souvent une étape contrôlée.

Cet après-midi



TP guidé — Provisionner et automatiser

Vous allez provisionner plusieurs ressources liées avec Terraform, explorer le graphe de dépendances généré, puis mettre en place un pipeline GitLab CI à plusieurs étapes qui valide votre code à chaque push.